Qt для тех, кто делал веб: как читать этот код

Почти у каждого понятия Qt есть близкий аналог в вебе. Ниже — словарик, потом путь одного клика через код, потом минимум C++, без которого не прочитать файлы.

1. Словарик «веб → Qt»

Веб	Qt	Где у нас
DOM-элемент, компонент React	**`QWidget`** (кнопка `QPushButton`, надпись `QLabel`, поле `QSpinBox`…)	все `pages/`, `ui/`
Дерево DOM (родитель → дети)	Дерево виджетов: у каждого виджета есть родитель	—
Удалили узел — пропало и всё внутри	Удалили родителя — Qt сам удалит детей. Поэтому `new` без `delete` — это нормально	`new QLabel` в `ui/widgets.cpp`
`display: flex; flex-direction: row / column`	**`QHBoxLayout` / `QVBoxLayout`**	наши `hbox({...})`, `vbox({...})` в `ui/widgets.cpp`
`flex-grow: 1`	число растяжения: `{widget, 1}`	`hbox({left_, {middle_, 1}, right_})` — середина забирает всё свободное место
пустой `<div style="flex:1">`	`Item::spring()`	расталкивает соседей
`gap`, `padding`	`spacing`, `margins` (последние аргументы `hbox`/`vbox`)	—
`min-width`, `width: fixed`	`setMinimumWidth`, `setFixedWidth`, «политика размера» `setSizePolicy`	`DemoPage::fit_box`
CSS	**таблица стилей** `setStyleSheet(...)` — урезанный CSS	`ui/style.cpp`, функция `init_style`
`id` / `class` в CSS	`setObjectName("card")` → селектор `QFrame#card`	`card()` в `ui/widgets.cpp`
`addEventListener('click', ...)`	**`connect(кнопка, &QPushButton::clicked, ...)`** — «сигнал → слот»	везде в конструкторах

Веб	Qt	Где у нас
`new CustomEvent(...)` + `dispatchEvent`	свой сигнал: объявить в `signals:`, послать `emit changed(42)`	`ParamControl::changed`, `RunControls::tick`
цикл событий браузера	`app.exec()` в `main.cpp` — та же идея: не блокировать надолго	—
`setTimeout` / `setInterval`	**`QTimer`** (`singleShot` — один раз)	кадры в `RunControls::frame`
`<canvas>` + `ctx.fillRect`, `ctx.lineTo`	**`paintEvent`** + **`QPainter`** (`drawRect`, `drawPolyline`, `drawEllipse`)	все графики в `ui/plots/`
`requestAnimationFrame` → перерисовка	`update()` — «перерисуй меня, когда удобно»; Qt позовёт `paintEvent`	`TimePlot::update_view`
`window.onresize`, `mousedown`, хуки жизненного цикла	переопределённые методы: `resizeEvent`, `mousePressEvent`, `showEvent`	`DemoPage::resizeEvent`, `BoxView::mousePressEvent`
роутер / одна видимая страница	`QStackedWidget` — стопка страниц, видна одна	`MainWindow` (`main_window.cpp`)
`innerHTML` с `<b>`, `<sub>`	`QLabel` понимает маленький кусок HTML	`ratio_text`, `stats_text`
статические файлы, бандлер	**ресурсы** `resources.qrc`: картинки и HTML вшиваются в exe, путь `":/assets/..."`	`ui/icons.cpp`, `theory/section_view.cpp`
сборка (webpack, `npm run build`)	**CMake** (`CMakeLists.txt`), команда `make`	`Makefile`
транспиляция JSX	**moc** — генератор кода для сигналов; нужен макрос `Q_OBJECT` в классе	`ParamControl`, `DemoPage`…

2. Как устроен экран — на примере

Карточка «Газ» (`pages/demo/gas_panel.cpp`, упрощённо):

```
temperature = new ParamControl("Температура T, K", 50, 1500, 300);   //
<input type=range> + число
n_total     = new ParamControl("Число молекул N", 2, 300, 150);

setLayout(vbox({card({gas_mode, temperature, n_total, mass_a}, "Газ")}));
```

```
//            ^ столбец  ^ белая карточка с заголовком, внутри — элементы
столбиком
```

В вебе это было бы:

```
<div style="display:flex; flex-direction:column">
  <div class="card"><h3>Газ</h3> <ParamControl .../> <ParamControl .../>
...</div>
</div>
```

Вся страница «Презентация» (`DemoPage`) — это три колонки:

```
hbox({left_, {middle_, 1}, right_})   // сосуд | графики (растягиваются) |
настройки
```

3. Путь одного клика: «▶ Старт»

```
QPushButton "Старт"  ──clicked──▶  RunControls::start()
(pages/demo/run_controls.cpp)
                                     ├─ emit started()   ──▶
DemoPage::on_start()   (demo_page.cpp)
                                     │                          └─
exp.rebuild()        новый газ (model/experiment.cpp)
                                     └─ timer.start(16 мс)
QTimer  ──timeout (каждые 16 мс)──▶  RunControls::frame()
                                     └─ emit tick(fast, factor) ──▶
DemoPage::on_tick()
                                                                  ├─
exp.advance_by(...)   физика (model/)
                                                                  └─
refresh() ─▶ каждый график: update_view(...) → update()
Qt, когда удобно  ──▶  TimePlot::paintEvent() → draw_data(QPainter&)
рисует, как на <canvas>
```

Связи («кто кого слушает») собраны в конце конструктора `DemoPage`, блок `// --- связи ---`:

```
connect(run, &RunControls::started, this, &DemoPage::on_start);   //
run.addEventListener('started', this.on_start)
connect(run, &RunControls::tick,    this, &DemoPage::on_tick);
```

4. Минимум C++, чтобы читать

C++	JS/TS	Пример
`.h` — объявления (что есть у класса), `.cpp` — код	как `.d.ts` + `.js`	`graph_panel.h` / `graph_panel.cpp`
`class A : public QWidget`	`class A extends HTMLElement`	—
`auto* x = new QLabel;`	`const x = document.createElement(...)`	`*` — указатель (ссылка на объект)
`x->setText("...")`	`x.setText("...")`	`->` для указателя, `.` для обычного объекта
`[this] { start(); }`	`() => this.start()`	лямбда (стрелочная функция)
`[this](int n) { ... }`	`(n) => { ... }`	лямбда с параметром
`std::vector<double>`	`number[]`	—
`std::map<Key, Value>`	`Map`	`exp.hist`
`std::optional<double>`	`number | undefined`	текущее значение может отсутствовать
`const Experiment& exp`	параметр «только читать, без копирования»	—
`namespace settings { ... }`	модуль / объект с константами	`settings::FONT_SCALE`
`constexpr double X = 1.5;`	`const X = 1.5` (известно при сборке)	`settings.h`
`QString::fromUtf8("...")`	обычная строка (Qt хранит текст по-своему)	все русские надписи
`_` в конце имени (`ratio_`)	наше соглашение: «приватное поле класса»	—

5. В каком порядке читать

1. `src/settings.h` — все числа, которые можно крутить.
2. `src/ui/widgets.cpp` — наши «кирпичики» (`label`, `button`, `hbox`, `vbox`, `card`, `ParamControl`): всё остальное собрано из них.
3. `src/pages/demo/demo_page.cpp` — главная страница: конструктор = вёрстка + связи, ниже — обработчики событий.
4. Один график, например `src/ui/plots/time_plot.cpp`: `update_view` (получить данные) → `draw_data` (нарисовать).
5. Модель (`src/model/`) — обычный C++ без Qt, физика: `physics.cpp` → `experiment.cpp`.

Общая карта файлов и «какой вопрос — какой файл» — в `ARCHITECTURE.md`, быстрые правки — в `ПРАВКИ.md`.